

MLP FUNDAMENTALS

Perceptron  $o = f(w_0 + \sum_i w_i x_i)$ : aggregation is a SUM of PRODUCTS (dot product, NOT cross/Cartesian);  $f = \text{step} \Rightarrow \text{binary 0/1}$  (NOT "0 to 1"=sigmoid).

**XOR**: a single perceptron CAN'T do it — XOR is **not linearly separable** (one line can't split it); needs a hidden layer.

**Why MLP is powerful**: non-linear activations stacked over hidden layers (universal approx). *Stacked linear layers collapse to ONE linear map* — depth without nonlinearity adds nothing.

**Linear algebra**: weights = MATRIX (rows in, cols out), bias = VECTOR added after matmul:  $a = f(xW + b)$ .

**Pipeline (match)**: (1) forward = activations layer-by-layer; (2) error = output vs target via cost; (3) backward = chain-rule gradients (direction+magnitude); (4) update = move OPPOSITE gradient  $\times$  lr. *Trap: "gradient gives direction" ("backward")  $\neq$  "params moved opposite gradient" (update).*

Q15 PERCEPTRON STEP TRUTH TABLE

$y = H(Wx + b)$ ,  $H(z)=1$  if  $z \geq 0$  else 0 (boundary  $z=0 \rightarrow 1$ ).

**Zero-COLUMN**: col  $j$  all  $0 \Rightarrow x_j$  ignored  $\Rightarrow$  only 2 (#live cols) codes (NOT  $2^n$ ).

**Zero-ROW**: row  $i$  all  $0 \Rightarrow z_i = b_i$  const  $\Rightarrow y_i$  locked (1 if  $b_i \geq 0$ ).

$Ex\ W = \begin{pmatrix} 1 & 0 \\ -1 & 0 \end{pmatrix}, b = \begin{pmatrix} -5 \\ -1.5 \\ 2.5 \end{pmatrix}$ : col2=0  $\Rightarrow$  only  $x_1$ .

$x_1=0 \Rightarrow (0, 0, 1)$ ;  $x_1=1 \Rightarrow (1, 0, 1)$ . Reachable =  $\{(0, 0, 1), (1, 0, 1)\}$ ;  $y_2=0, y_3=1, y_1=x_1$ . **Update**:  $dw = y(p - \text{step } o)$ ;  $\Delta w_{ij} = \eta x_i x_j$ ,  $\Delta b = \eta e$  (bias input=1),  $w \leftarrow w + \Delta w$  (ADD).

Q7 BACKPROP — "connect the derivative"

A layer's **error** = how much the loss moves when that layer's weighted-sum moves. Build it from THREE things multiplied:

**error here** = (next layer's weights)  $\times$  (next layer's error)  $\times$  (this activation's slope).

**Match-key (the exam Q)**:

- loss vs. NEXT weighted-sum" = the next layer's error.
  - "NEXT weighted-sum vs. this output" = the next layer's weights.
  - "this output vs. this weighted-sum" = the activation's slope (sigmoid = out $\times$ (1-out)).
- Weight gradient** = (this layer's error)  $\times$  (input slope below). **Bias gradient** = the error itself. **Trap**: swapping "next error"  $\leftrightarrow$  "next weights" (weights = the linear part, error = the loss part); take the slope at the weighted-sum, not the output. Student 0.33. (*Greek form*  $\rightarrow$  formula list.)

DERIVATIVE DICTIONARY (plain: "diff THIS, get THAT")

**A linear step** (weights $\times$ input+bias), diff w.r.t. the **input** = the **weights**; w.r.t. the **weights** = the **input**; w.r.t. **bias** = 1.

**An activation**, diff w.r.t. its input = its **slope**:

sigmoid = out $\times$ (1-out); tanh = 1-out<sup>2</sup>; ReLU = 1 if input $\geq 0$  else 0; linear = 1.  
**A loss**, diff w.r.t. the prediction: MSE = 2 $\times$ (pred-true); sigmoid+BCE = pred-true; softmax+cross-entropy = pred-true (the clean one). **A layer's error** = (next weights)  $\times$  (next error)  $\times$  (this slope). **Last-layer error** = slope  $\times$  loss-derivative (= pred-true for softmax+CE). **Weight gradient** = layer-error  $\times$  input-from-below; **bias gradient** = layer-error.

**RNN**: same weights reused each step  $\Rightarrow$  ADD up the contributions over all time steps. *One rule: diff a linear thing  $\rightarrow$  weights; diff an activation  $\rightarrow$  its slope; the loss-side piece is the next layer's error.*

ACTIONS + DERIVATIVES

sigmoid  $\frac{1}{1+e^{-z}}$ ,  $\sigma' = \sigma(1-\sigma)$  (max .25@0), (0, 1);

tanh  $(-1, 1)$ ,  $1 - \tanh^2$ ; ReLU max(0, z), 1[z $\geq 0$ ];

LeakyReLU  $\alpha z$  if  $z \leq 0$ ; softmax  $\frac{e^{-z_i}}{\sum_j e^{-z_j}}$  ( $\geq 0$ , sum 1).

**ReLU beats vanishing**: grad is constant 1 for  $z > 0$  (non-saturating), NOT "because nonlinear". **Dying ReLU** ( $x \in (0, 1)$ ,  $W \geq 0$ ): dies if  $Wx + b \leq 0 \forall x \Rightarrow$  max bias  $v=0$ , max weight  $\alpha=0$ .

**Softmax in hidden layer**: backprop STILL works (differentiable); bad only b/c it forces competition (sum-to-1), limiting features.

**Mean over U**(-1, 1): sig .5, tanh 0, ReLU .3, LeakyReLU(.4) .2 (slope nearer 1  $\Rightarrow$  mean nearer 0). **BCE vs MSE (binary)**: use sigmoid+BCE;  $\frac{\partial L}{\partial z} = -o-t$ . MSE+sigmoid = tiny/flat gradients.

Multi-task loss = SUM  $L_1 + L_2$ .

Q9 LAYER-NORM NUMERIC (per ex., the ROW)

$\mu = \frac{1}{N} \sum x_i$ ,  $\sigma^2 = \frac{1}{N} \sum (x_i - \mu)^2$  (avg sq dev,  $\div N$ ),

$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ ,  $y_i = \gamma \hat{x}_i + \beta$ .

$Ex\ [2, 2, 6, 6], \gamma=1.5, \beta=-.5, \epsilon=0$ :  $\mu=4$ ,  $\sigma^2 = \frac{1}{4}(4+4+4+4)=4$ ,  $\hat{x}_3 = \frac{6-4}{2}=1$ ,

$y_3 = 1.5(1) - .5 = \boxed{1}$ .

**Trap**:  $\div N$  not  $N-1$ ; report  $\sigma^2=4$  NOT std(2); "3rd moment"=value 6. Student 0.25.

**Batch-Norm numeric**: same formula but  $\mu_j, \sigma_j^2$  DOWN feature- $j$ 's COLUMN across batch. Axis flip is the question.

Q6 NORMALIZATION: BN/LN/IN/GN

Same formula, differ by POOLED axis. Tensor (N, C, H, W):

**BN** pool (N, H, W)/chan (batch-dep, big batch, conv+FC).

**LN** pool (C, H, W)/sample (feats of 1 ex; transf/RNN).

**IN** pool (H, W)/(sample,chan). **GN** (H, W) per channel-group.

**Close-key (Q6)**: B1=Batch, B2=mini-batch (NOT "layers"), B3=Layer, B4=features, B5=Group, B6=noisy (NOT "slow"), B7=Instance, B8=Batch. **Trap**: tiny batch  $\Rightarrow$  BN NOISY (not slow); "features of ONE sample"=LN not IN.

REGULARIZATION (whole quiz topic)

**Goal**: close the generalization gap (val-train), NOT raise train acc/speed.

**L2/weight decay**  $L + \frac{\lambda}{2} \sum w^2$ , step

$w \leftarrow w - \eta(\nabla + \lambda w)$ , shrinks weights, lcapacity  $\Rightarrow$  train acc DROPS, val acc RISES. **L1**  $\lambda \sum |w|$  = sparsity.

**Dropout**: zero each unit w.p.  $p$  in **TRAIN only**, off/scaled at test;  $\hat{h} = \frac{1}{p}(m \odot h)$ . *Lower*  $p$  0.5 $\rightarrow$ 0.3 = *LESS* reg. Dropout IS regularization.

**Data aug**: more diversity = reg; must keep label (no-flip Q/QK for images, wrong for audio/spectrogram).

**BatchNorm**: primary role = training stability/speed (smoother grads, higher lr); only mild reg side-effect.

**Bias-variance**: high bias = restrictive, MISSES relations (underfit); high variance = fits noise (overfit).  $E[(y-f)^2] = \text{Bias}^2 + \text{Var} + \sigma^2$ .

**Capacity from curves**: lower TRAIN err = higher capacity; the regularizer is the param that RAISES train err but NARROWS the gap. Too much reg  $\Rightarrow$  underfit (both rise).

OPTIMIZERS (in words)

Always step the OPPOSITE way to the gradient: new = old - lr $\times$ gradient. (pred too high  $\Rightarrow$  gradient  $\uparrow \Rightarrow$  step down.)

**Momentum**: keep a running "velocity" (a blend of past gradients) and step by that — smooths, speeds up. **Nesterov**: same, but look ahead first (measure the gradient AFTER the momentum step).

**AdaGrad**: divide the step by the running SUM of squared gradients — but that sum only grows, so the rate shrinks to zero and stalls. **RMSProp**: use a decaying average instead (forgets old), so it stays usable.

**Adam** = momentum + RMSProp: track a running average of the gradient (1st moment) AND of the squared gradient (2nd moment), bias-correct both early on, step = lr $\times$ avg-grad/ $\sqrt{\text{avg-sq-grad}}$ .

Defaults .9, .999,  $10^{-8}$ .

**Traps**: Adam tracks the 1st & 2nd MOMENTS (NOT "weight & bias"); the  $\epsilon$  is just to avoid divide-by-zero; lr too high = overshoot, too low = slow; Adam is still first-order (not Newton). grad.zero() because gradients add up by default. (*Greek forms*  $\rightarrow$  formula list.)

Q11 PARAM & TRAINING-MEMORY COUNT

params/layer = in-out+out. stored/iter =  $P \times m$ : plain SGD 2, Mom/Nesterov/RMSProp/AdaGrad 3, Adam 4.

$Pin\ 4 \rightarrow 3 \rightarrow 2$ :  $P=(12+3)+(6+2)=23$ ; mom  $3P=69$   $\checkmark$ .

$AnS\ 8 \rightarrow 5 \rightarrow 3 \rightarrow 2$ :  $P=45+18+8=\boxed{71}$ . Nesterov

$3P=\boxed{213}$ ; Adam  $4P=\boxed{284}$ .

**Trap**: bias  $\uparrow$ out ONCE/non-input layer; SGD NO buffer; Nesterov 1 buffer (not 4). Student 0/1.

INIT + HYPERPARAMS

**Init**: tanh/sigmoid  $\rightarrow$  Xavier Var= $\frac{1}{n_{in}}$  (stops

saturation); ReLU  $\rightarrow$  He  $\frac{2}{n_{in}}$ , b=0 (2 offsets ReLU killing half). *Swap=trap*.

**Hyperparams** (lr, #layers, hidden, batch, epochs,  $\lambda$ ): set BEFORE training, NOT learned; tuned on val. (params=weights/biases, learned.)

**Search**: grid = exhaustive, cost  $\prod n_i$  (exp); random = cheaper, more distinct values, often faster; Bayesian = guided by past, sample-efficient.

WORKED MICRO-NUMERICS (drill)

**Softmax** [2, 0, -1]:  $e^2 \approx 7.4, 1, .37$ ; sum $8.76 \Rightarrow$  [.84, .11, .04].

**CE** (true class,  $p=.84$ ):  $-\ln .84 \approx 0.17$ . **GD step** ( $\theta 1, \nabla .4, \eta .5$ ) = 0.8.

**Adam**  $t=1$  ( $g .2, \alpha .1, \theta .5$ ):  $m_1=.02, v_1=4e-5$ ,

$\hat{m}_1=.2, \hat{v}_1=.04, \hat{v}_1 \div m_1=.5-1 \cdot \frac{.2}{.2}=0.40$ .

LOSS FUNCTIONS + softmax-CE gradient

**MSE**  $\frac{1}{N} \sum (\hat{y} - y)^2$  (regression). **MAE**  $\frac{1}{N} \sum |\hat{y} - y|$  (robust to outliers).

**BCE**  $-\frac{1}{N} \sum [y \ln p + (1-y) \ln (1-p)]$  (binary+sigmoid).

**CE**  $-\sum y_i \ln p_i$  (multiclass+softmax) = LogSoftmax+NLL.

**softmax+CE grad** =  $p-y$  (clean — why we pair them); sigmoid+BCE grad =  $o-t$ . **Hinge** max(0,  $1-y\hat{y}$ ); **KL** for distributions.

PyTorch: pass RAW logits to CrossEntropyLoss (no softmax); BCEWithLogitsLoss folds sigmoid.

VANISHING / EXPLODING GRADIENTS

Deep nets multiply many Jacobians  $\Rightarrow$  product  $\rightarrow 0$  (vanish) or  $\rightarrow \infty$  (explode).

**Vanish**: sigmoid/tanh saturate ( $\sigma' < .25$ ). **Fixes**: ReLU (const grad), Xavier/He init, BatchNorm, residual/skip, LSTM/GRU gating.

**Explode fixes**: gradient clipping, smaller lr, normalization.

**RNN**: repeated  $W_{hh}$  multiply  $\Rightarrow$  vanish over long seqs; gating + clip fix it.

GD VARIANTS + CHAIN-RULE GROUP-ING

**Batch** = full data/step (stable, slow). **SGD** = 1 sample (noisy, escapes minima). **Mini-batch** = typical (GPU-friendly).

**Grouping (Q-type)**:

$\frac{\partial L}{\partial z_j - 1} = \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial a_{j-1} - 1} \frac{\partial a_{j-1}}{\partial z_j - 1}$ ; first TWO = error at

$= \frac{\partial L}{\partial a_j} \frac{\partial a_j - 1}{\partial a_{j-1} - 1}$

the ACTIVATION ( $dL/da$ ); third = local act deriv (don't call the pair the full  $\delta$ ).

XOR + UNIVERSAL APPROXIMATION

XOR truth table:

(0, 0) $\rightarrow$ 0, (0, 1) $\rightarrow$ 1, (1, 0) $\rightarrow$ 1, (1, 1) $\rightarrow$ 0. Not linearly separable  $\Rightarrow$  needs  $\geq 1$  hidden layer; hidden units remap so output can linearly split.

**Universal approximation**: an MLP with 1 hidden layer + non-linear activation can approximate any continuous function (enough units). Depth = exponentially more efficient than width. Remove nonlinearity  $\Rightarrow$  collapses to linear.

SOFTMAX+CE DERIVATION (why  $p-y$ )

$L = -\sum_k y_k \ln p_k$ ,  $p_k = \frac{e^{-z_k}}{\sum_j e^{-z_j}}$ .

$\Rightarrow \frac{\partial L}{\partial z_j} = p_i - y_i$  (softmax Jacobian  $\times$  CE cancel)

cleanly).

One-hot  $y \Rightarrow$  gradient = *predicted* - *true*. Same clean shape as sigmoid+BCE ( $o-t$ ) — that pairing is why these losses give strong, non-vanishing gradients.

Q13 PyTorch CODE-READING RULEBOOK

**R1 self.** self.x $\rightarrow$ nn.Linear(a,b) in .init...; bare x $\rightarrow$  (no self.)=LOCAL  $\neq$  not in parameters(), not optimized, not .to(dev); self.x(b)  $\rightarrow$  AttributeError.

**R2 apply.** self.head(b) runs fwd; self.head alone = Module, sigmoid(self.head)=BUG  $\rightarrow$  sigmoid(self.head(h)).

**R3** every self.x in forward must exist in init.

**R4 "X can't be used in Y-net"=distractor**: dropout/shared trunk/BN/conv work in ANY net; sigmoid OK binary.

**R5 list** $\rightarrow$ nn.ModuleList: plain [...] RUNS but params unregistered (silent, no crash, no train).

**R6 loss**: CrossEntropyLoss=LogSoftmax+NLL on RAW LOGITS (NO softmax in model); target Long idx (labels.float() $\rightarrow$ RuntimeError: expected Long got Float).

**R7 loop**: zero.grad  $\rightarrow$  model(x)  $\rightarrow$  loss=crit  $\rightarrow$  backward (gradients  $\rightarrow$  step update).

**Q13 CustomerNet**! TRUE bugs {1, 2, 4}: (2) reg.head $\neq$  no self.=unregistered; (4) sigmoid(self.class.head); (1)=(4) restated (tick BOTH). FALSE: shared/sigmoid/dropout OK. Student {2, 4}=0.67.

PyTorch BUILDING BLOCKS

nn.Linear =  $y = xW^T + b$ , affine ONLY (no activation). .to(device) moves params+buffers+inputs to CPU/GPU (model & data same device); NOT freeze/././compile.

view(-1,784) flattens 28 $\times$ 28 b/c first FC needs 1-D ( $-1$  infers batch); NOT for SGD/CE/overfit.

optim.step() applies update from grads; backward() computes grads (don't swap). zero.grad() else accumulate.

Epoch avg loss = total\_loss/len(loader).

Q2/Q4/Q10 CNN

Out  $O = \lfloor \frac{W-F+2P}{S} \rfloor + 1$  per axis; "same"  $P = \frac{F-1}{2}$ .

Conv2d params  $C_{out}(C_{in} K^2 + 1)$ ;

pool/ReLU/flatten 0, flatten in =  $C_{out} H_{out} W_{out}$ .

Ex 28  $\frac{k3p1}{28} \frac{pool2}{28} \rightarrow 14, 8.14-14=1568$ .

**Q2 MLP**: D3=filter, D2=low-level, D8=CNN, D5=match, D7=weight-share, D9=feature-map; NO match D1,D4,D6.

**Q4 seq**: enc dec sample $\rightarrow$ dec upsample+skip; out  $H \times W \times C_{cls}$ . KEEP grid, CHANGE channels, NEVER flatten. WRONG = {3, 5, 6, 8}.

**Q10 lines**: L9 Conv2d; L10 MaxPool=shrink; L15 flatten; L26 zero-grad; L27 model(images)=preds; L28 criterion(,labels.float())=Long-vs-Float error

**HERE**; L29 backward; L30 step.

CNN MECHANICS (quiz traps)

MaxPool default stride = kernel (not 1!):

28  $\frac{conv5}{28} \frac{pool2}{28} \rightarrow 12$ .

What CNNs compute = **cross-correlation** (kernel NOT flipped).

conv = LEARNABLE kernel; pool = FIXED agg, NO params.

**equivariant $\neq$ invariant**:

conv=translation-equivariant (shift in $\rightarrow$ shift out); invariance from POOLING; NO rotation guarantee.

**Param sharing**: same kernel at every position  $\Rightarrow$  fewer params + equivariance.

**Edge kernels**: cols  $(-1, 0, +1)$  = vertical =  $\partial I/\partial x$ ;

rows = horizontal =  $\partial I/\partial y$  (detects edges  $\perp$  to weight variation).

Conv (kernel=full  $W \times H$ )  $\equiv$  Flatten+Linear.

Save compute  $\Rightarrow$  INCREASE stride (risk: skip detail). AdaptiveAvgPool1d(1) = fixed size any length.

Multi-objective: minimize  $L_1$ , maximize  $L_2 \Rightarrow L = L_1 - L_2$  (subtract to maximize).

Q5/Q12/Q8 RECURRENCE

$RNN\ h_t = f(W_{hh} h_{t-1} + W_{xh} x_t + b)$ : depends on BOTH input & prev state, shared weights every step. tanh = nonlinearity + bound  $[-1, 1]$ .

**GRU order (Q5)**: (1) compute gates  $z, r \rightarrow$  (2) reset  $r$  filters old  $h \rightarrow$  (3) candidate  $\rightarrow$  (4) update  $z$  blends old+new. ( $r$  sits INSIDE the candidate; blend is LAST.)  $r$ =forget past,  $z$ =convex blend (fixes vanishing). *concat dim = in+hid!* (cell equations  $\rightarrow$  code blocks)

**Q12 key**: combined=14, gates/cand Linear(14,6), **essential** Linear(2,3), softmax dim 1,

**accres** =  $T = BPTT=5$ .

**Q8=E**: gating regulates state OVER TIME (vanishing); attention reads ALL enc states (fixed-context bottleneck). Complementary; traps SWAP.

RNN/SEQ2SEQ DEEP

**BPTT**: unroll,  $dJ/dW$  = SUM over paths (shared  $W$ ):  $\frac{\partial J}{\partial \theta_2} \frac{\partial \theta_2}{\partial \theta_1} \frac{\partial \theta_1}{\partial w} + \frac{\partial \theta_2}{\partial \theta_1} \frac{\partial \theta_1}{\partial w} + \frac{\partial \theta_2}{\partial \theta_1} \frac{\partial \theta_1}{\partial w}$

(direct+indirect via  $h_1$ );  $h_0$  learned via BPTT.

**Task pairing**: regression  $\rightarrow$  Linear out+MSE; binary  $\rightarrow$ sigmoid+BCE;

multiclass  $\rightarrow$ softmax+categorical CE.

**CNN  $\rightarrow$  LSTM**: lstm\_init.size = last conv out\_channels; permute(0,2,1) to (B,seq,feat) out[,.,1]

seq steps: Linear(hid,classes)

**Seq2seq**: encoder  $\rightarrow$  final state  $\rightarrow$  decoder generates. **Teacher forcing** = feed

GROUND-TRUTH target.seq[t] (not own prediction) during training.

Q14 SINGLE-HEAD SELF-ATTENTION (enc)

Attn=softmax( $\frac{QK^T}{\sqrt{d_k}}$ ) $\cdot V$ .  $x(B,S,8)$ ;  $W_{qkv}$ :8 $\rightarrow$ 4;

$W_O$ :4 $\rightarrow$ 8;  $d_k=4$ .

**Template**:  $d_k, k, v = W_q(x), W_k(x), W_v(x)$ ;

$\text{scores} = \text{matmul}(q, k.\text{transpose}(-2, -1)) / (4 \cdot d_k) = /2$ ;

$\text{softmax}(\text{dim}=-1)$ ;  $\text{ctx} = \text{matmul}(a, v)$ ; out=Wo(ctx).

**Match-key**: (1) matmul(k,q,t) swap Q/K=RUNS; (2) matmul(v,a,w)=CRASH; (4) matmul(q,k) no .T=CRASH;

(5) /4=RUNS (wrong scale); (6) q $\cdot$ v.T=RUNS (V not K); (7) k $\cdot$ Wq(x)=RUNS; (8) dim=1=RUNS; (9)=correct;

(3)&(10)=NO code (decoys: mask / missing  $W_O$ ).

**RULE**: matmul runs  $\leftrightarrow$  last-dim(A)=2nd-last(B). Scale  $\sqrt{d_k} \neq 2$  NOT  $d_k=4$ .

ATTENTION DEEP (quiz)

**Role**: reweight tokens by relevance/context (NOT dim-reduction/copy/normalize).

$Q = XW_q$ ,  $K = XW_k$ ,  $V = XW_v$  map to a SHARED space. scores  $QK^T$  shape  $(B, S, S)$  (embed collapses), softmax over KEYS,  $z = \text{weights@V}$  = context blend.

**CNN CODE (annotated) — Q10**

```
conv1=nn.Conv2d(1,8,k=3,pad=1) > 1→8 CH, 28 STAYS
pool=nn.MaxPool2d(2,2) > HALVES 28→14
fc1=nn.Linear(8*14*14,10) > 1568→10
x=pool(F.relu(conv1(x))) > (B,8,14,14)
x=torch.flatten(x,1) > (B,1568)
x=fc1(x) > (B,10) RAW LOGITS
loss=CrossEntropyLoss()(x,y) > Y MUST BE LONG
> Y.FLOAT() → RUNTIMEERROR: EXPECTED LONG
GOT FLOAT
```

**GRU CELL CODE (annotated) — Q12**

```
> X,T,(B,8) H,(B,6) → CAT = (B,14)
z=sigmoid(Wz(cat[x,t,h])) > UPDATE GATE (B,6)
r=sigmoid(Wr(cat[x,t,h])) > RESET GATE (B,6)
h=torch.tanh(Wc(cat[x,t,r*h])) > CANDIDATE (B,6)
h=(1-z)*h+z*h <- BLEND OLD←NEW
a=softmax(atten(H),dim=1) > H:(B,5,6)→(B,5)
ctx=sum(a*H) > (B,6); CLF IN = CAT[CTX,H] = 12
> #SCORES = #STEPS = BPTT = 5
```

**MULTI-TASK nn.Module CODE (annotated) — Q13**

```
self.shared=nn.Linear(30,64) > SHARED TRUNK: OK
self.class_head=nn.Linear(64,1) > OK
reg_head=nn.Linear(64,1) > BUG2: NO
SELF.→UNREGISTERED
h=F.relu(self.shared(x))
purchase=sigmoid(self.class_head) > BUG4: NEEDS (H)
spends=self.reg_head(h) > (REG.HEAD DOESN'T EXIST)
> REAL BUGS = {1=EFFECT OF 4, 2, 4} — TICK
MECHANISM AND EFFECT
```

**PERCEPTRON UPDATE (Q1.10)**

```
error = target - output > PREDICTION ERROR
d.weights = learning.rate * error * inputs > WEIGHT
DELTA
d.bias = learning.rate * error > BIAS DELTA
weights = weights + d.weights > UPDATE WEIGHTS
bias += d.bias > UPDATE BIAS
```

**MLP BACKPROP (Q2.15)**

```
derivative_of_loss = (mlp.output - target) / (mlp.output*(1
- mlp.output)) > BCE→SIGMOID
delta_output = derivative_of_loss *
sigmoid.derivative(mlp.output,z) > ∂L/∂z
error_hidden = delta_output.dot(mlp.weights.hidden_output.T)
> CHAINRULE; [1,HIDDEN]
```

**MLP BACKPROP problem.type (Q2.16)**

```
delta_output = mlp.output - target > CLASS:
PRED-TARGET
delta_output = 2 * (mlp.output - target) > REGRESSION
MSE
error_hidden = delta_output.dot(mlp.weights.hidden_output.T)
> GRADIENT BACKWARD
delta_hidden = error_hidden *
mlp.activation.derivative(mlp.hidden,z) > LOCAL SLOPE
mlp.weights.hidden_output -=
mlp.hidden_output.T.dot(delta_output) > SHAPE
(HIDDEN,OUTPUT)
```

**ADAM OPTIMIZER (Q3.13)**

```
n = beta1* + (1-beta1)*g > FIRST MOMENT
v = beta2*v + (1-beta2)*(g**2) > SECOND MOMENT
n_hat = m / (1 - beta1**t) > BIAS CORRECTION
v_hat = v / (1 - beta2**t) > BIAS CORRECTION
update = lr*m_hat/(np.sqrt(v_hat)+eps) > PER-PARAM
SCALING
```

**CUSTOM SGD (Q3.14)**

```
with torch.no_grad(): > NO AUTOGRAD
param.data -= self.learning_rate * param.grad > SGD
UPDATE
param.grad.zero() > RESET GRADS
optimizer.step(params=MLP_model.parameters()) > ALL
PARAMS
```

**MLP TRAIN LOOP (Q4.7)**

```
MLP_model.to(device) > MOVE TO DEVICE
optimizer.zero_grad() > ZERO GRADIENTS
inputs = inputs.view(-1, 28*28) > FLATTEN IMAGE
outputs = MLP_model(inputs) > FORWARD PASS
loss = criterion(outputs, labels) > COMPUTE LOSS
```

loss.backward(); optimizer.step() > BACKPROP, UPDATE

**CONV FOR SEGMENTATION (Q5.4)**

```
self.L_n = nn.Conv2d(32, 64, (3,3)) > 64 CHANNELS
self.L_m = nn.Conv2d(64, 64, (3,3)) > 64 CHANNELS
cat(L_m, L_n) over channels > 128 > CONCAT AXIS
self.L_out = nn.Conv2d(128, 2, 1) > 1X1 CONV, 2
CLASSES
```

**CONV == FLATTEN+LINEAR (Q5.8)**

```
nn.Conv2d(NC_in, NC_out, (k,w, k,h)) > CONV PATH
nn.Flatten(0) ; nn.Linear(NL_in, NL_out) > FC PATH
equiv when k.w*W and k.h*H > KERNEL = FULL MAP
NL_in = C*W*H, NL_out = NC.out > MATCHING DIMS
```

**CONV+MAXPOOL SHAPE (Q5.10)**

```
nn.Conv2d(1, 32, (5,5)) > 28→24
nn.MaxPool2d(kernel_size=2) > STRIDE=2 DEFAULT
output dim = 12 > 24/2
pool stride defaults to kernel > THE GOTCHA
```

**FLATTEN -k LINEAR FEATS (Q5.11)**

```
input.shape = (128, 7, 7) > INPUT
nn.Conv2d(128, 32, (5,5)) > 7→3
nn.Linear(in.features*288, out.features*10) > 32*3*3=288
```

**1D CNN STRIDE+POOL (Q5.13)**

```
nn.Conv1d(128, 128, 3, stride=2) > INCREASE STRIDE
nn.AdaptiveAvgPool1d(1) > MISSING LINE
nn.Flatten(0); nn.Linear(128, 10) > 10 CLASSES
too-aggressive stride loses detail > THE RISK
```

**NUMPY 1D CONV (Q5.14)**

```
output.length = 98 > (100-3)+1
filters.shape = (4, 2, 3) > NFILT,C,K
seg = input[:,::1, i*s : i*s+k] > WINDOW SLICE
output[:,::1] = np.sum(seg*filters, axis=(-2,-1)) >
REDUCE C,K
```

**NUMPY 2D CONV (Q5.15)**

```
assert f.shape(0) == input.shape(0) > CHANNELS MATCH
output.h = (H - f.h) // stride + 1 > NO PADDING
patch = input[:, i*s:i*s+f.h, j*s:j*s+f.w] > ALIGNED
PATCH
output[:,i,j] = np.sum(patch*filters, axis=(1,2,3)) > DOT
PER FILTER
```

**CNN DROPOUT (Q6.13)**

```
self.dropout = nn.Dropout(p=dropout_prob) > DROPOUT
LAYER
x = self.dropout(x) > AFTER EACH CONV
x = x.view(-1, self.fc1.in.features) > FLATTEN
```

**DROPOUT+AUGMENT (Q6.14)**

```
model.dl = SimpleCNN.dropout(dropout_prob=0.3) > WEAKER
REGULARIZATION
transforms.RandomRotation(degrees=30) > AUGMENTATION
= REGULARISATION
optimizer = optim.Adam(model.dl.parameters(), lr=0.001) >
ADAPTIVE OPTIMIZER
transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5)) >
NUMERICAL STABILITY
```

**CNN-LSTM (Q7.6)**

```
self.conv3 = nn.Conv1d(128, 256, kernel_size=3, stride=1,
padding=1) > LAST CONV OUT
self.lstm = nn.LSTM(input_size=256, hidden_size=hidden_dim,
num_layers=num_layers, batch_first=True) > IN=256
self.fc = nn.Linear(hidden_dim, num_classes) > FC DIMS
for layer in [self.conv1, self.conv2, self.conv3]: > CONV
LOOP
x = relu(layer(x)) > RELU APPLY
x = x.permute(0, 2, 1) > PERMUTE LSTM
out, _ = self.lstm(x) > RUN LSTM
out = out[:, -1, :] > LAST STEP
```

**GRU CELL (Q7.11)**

```
self.W_z = nn.Linear(input_size + hidden_size, hidden_size)
> GATE (24→16)
combined = torch.cat((x, h_prev), dim=1) > CONCAT
z = torch.sigmoid(self.W_z(combined)) > UPDATE GATE
r = torch.sigmoid(self.W_r(combined)) > RESET GATE
```

```
combined.reset = torch.cat((x, r * h_prev), dim=1) >
APPLY RESET
h.tilde = torch.tanh(self.W_h(combined.reset)) >
CANDIDATE
h.next = (1 - z) * h_prev + z * h.tilde > INTERPOLATE
```

**SIMPLE RNN (Q7.13)**

```
self.h0 = nn.Parameter(torch.rand([1, hidden_size])) >
LEARNABLE INIT
self.Wxh = nn.Linear(input_size, hidden_size) > INPUT
MAP
self.Whh = nn.Linear(hidden_size, hidden_size) >
RECURRENT MAP
self.Why = nn.Linear(hidden_size, output_size) > OUTPUT
MAP
h,t = self.tanh(self.Wxh(x) + self.Whh(h_prev)) >
RECURRENCE
y,t = self.Why(h,t) > OUTPUT
```

**GRU FULL (Q7.14)**

```
r,t = self.sigmoid(self.Wxr(x) + self.Whr(h_prev)) > RESET
GATE
z,t = self.sigmoid(self.Wxz(x) + self.Whz(h_prev)) >
UPDATE GATE
h.candidate = self.tanh(self.Wxh(x) + r,t *
self.Whh(h_prev)) > FILTERED CANDIDATE
h,t = (1 - z,t) * h_prev + z,t * h.candidate > CONVEX
COMBO
y,t = self.Why(h,t) > OUTPUT
```

**SEQ2SEQ (Q7.15)**

```
self.encoder = nn.RNN(input_size, hidden_size) > ENCODER
self.decoder = nn.RNN(input_size, hidden_size) > DECODER
-, hidden = self.encoder(input_seq) > ENCODE
decoder_input = torch.zeros(1, len(vocab)) > INIT
DECODER IN
decoder_output, decoder_hidden = self.decoder(decoder_input,
decoder_hidden) > DECODE STEP
decoder_input = target_seq[t].view(1, -1) if teacher.force
else softmax(output,-1) > TEACHER FORCING
```

**ATTENTION EINSUM (Q8.4)**

```
energy = torch.einsum("nqhd,nkhd->nhqk", [queries, keys]) >
PER-HEAD QK SCORES
energy /= queries.size(-1) ** 0.5 > SCALE
attention = torch.nn.functional.softmax(energy, dim=3) >
SOFTMAX
out = torch.einsum("nhll,nlhd->nlhd", [attention, values])
> WEIGHT VALUES
out = out.reshape(N, query.len, self.heads * self.head_dim)
> MERGE HEADS
out = self.fc.out(out) > FINAL PROJ
```

**SELF-ATTENTION (Q8.14)**

```
self.query = nn.Linear(embed_dim, embed_dim) > Q,K,V PROJ
Q = self.query(x); K = self.key(x); V = self.value(x) >
PROJECT
scores = torch.matmul(Q, K.transpose(-2, -1)) /
(self.embed_dim ** 0.5) > RAW SCALED LOGITS
weights = F.softmax(scores, dim=-1) > (B,SEQ,SEQ)
context = torch.matmul(weights, V) > WEIGHTED VALUES
```

**MULTI-HEAD ATTENTION (Q8.15)**

```
self.head_dim = embed_size // heads > PER-HEAD DIM
assert self.head_dim * heads == embed_size, "Embedding size
needs to be divisible by heads" > BLANK1
self.values = nn.Linear(self.head_dim, self.head_dim,
bias=False) > K,Q SAME
self.fc.out = nn.Linear(heads * self.head_dim, embed_size) >
OUTPUT PROJ
input_queries = input_queries.reshape(N, query.len,
self.heads, self.head_dim) > BLANK2 SPLIT HEADS
energy = torch.einsum("nqhd,nkhd->nhqk", [queries, keys]) >
BLANK3 SCORES
```

**SINUSOIDAL POS ENCODING (Q9.5)**

```
div_term=torch.exp(torch.arange(0,d_model/2).float()*(−math.
pi/(10000.0/d_model)) )
> FREQ TERM
pe[:,0::2]=torch.sin(position*div_term) > EVEN=SIN
pe[:,1::2]=torch.cos(position*div_term) > ODD=COS
```

**POSITIONALENCODING MODULE (Q9.7)**

```
position=torch.arange(max_len).unsqueeze(1) > BLANK
UNSQEEZE
```

```
freq=(10000.0**(-2*torch.arange(0,d_model/2)/d_model)).unsque
ze(0)
> FREQ TERM
pe[:,0,0::2]=torch.sin(position*freq) > BLANK SIN
pe[:,0,1::2]=torch.cos(position*freq) > ODD=COS
x+=self.pe[:,x.size(0)] > ADD PE
```

**FORMULAS (127)**

**MLP/Perceptron.**  $z = \sum_{i=1}^n w_i x_i + b$  ♦  
 $\hat{y} = f(\sum_i w_i x_i + b)$  ♦  $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$  ♦  
 $a^{(l)} = f(z^{(l)})$  ♦  $y = xW^T + b$  ♦  
 $\Delta w_i = \eta (y - \hat{y}) x_i$  ♦  $\Delta b = \eta (y - \hat{y})$  ♦

**Activations.**  $f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$  ♦

$\sigma(z) = \frac{1}{1+e^{-z}}$  ♦  $\sigma'(z) = \sigma(z) (1 - \sigma(z))$  ♦

$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ ,  $\tanh'(z) = 1 - \tanh^2(z)$  ♦

$\text{ReLU}(z) = \max(0, z)$  ♦  $f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$  ♦

$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$  ♦  
 $w x + b \leq 0 \Rightarrow b \leq 0, w \leq -b$  ♦  
**Loss.**  $L = \frac{1}{N} \sum_n (y_n - \hat{y}_n)^2$  ♦  
 $\frac{\partial L_{\text{MSE}}}{\partial o} = 2(o - t)$  ♦  $L = -\sum_i y_i \log \hat{y}_i$  ♦  
 $L = -z y + \log \sum_j e^{z_j}$  ♦  $\frac{\partial L_{\text{BCE}}}{\partial o} = \frac{o-t}{o(1-o)}$  ♦  
 $\frac{\partial L}{\partial z_{\text{out}}} = o - t$  ♦  $L_{\text{total}} = L_1 + L_2$  ♦  
**Backpropagation.**  $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$  ♦  
 $\delta^{(L)} = \nabla_a L \odot f'(z^{(L)})$  ♦  
 $\delta^{(l)} = \left( W^{(l+1)} \right)^T \delta^{(l+1)} \odot f' \left( z^{(l)} \right)$  ♦  
 $\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$  ♦  $\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$  ♦  
 $\frac{\partial z^{(l)}}{\partial w^{(l)}} = a^{(l-1)}$  ♦  $\frac{\partial L}{\partial w} = \frac{1}{N} \sum_{n=1}^N \frac{\partial L_n}{\partial w}$  ♦

**Optimizers.**  $\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$  ♦  
 $\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} L(\theta; x^{(i)}, y^{(i)})$  ♦  
 $g = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L_i$  ♦  $v_t = \beta v_{t-1} + (1 - \beta) x_t$  ♦  
 $v_t = \beta v_{t-1} + (1 - \beta) g_t$ ,  $\theta_{t+1} = \theta_t - \alpha v_t$  ♦  
 $v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t - \alpha \beta v_{t-1})$  ♦  
 $r_t = r_{t-1} + g_t^2$ ,  $\theta_{t+1, i} = \theta_{t, i} - \frac{\alpha}{\sqrt{r_{t, i} + \epsilon}} g_{t, i}$  ♦  
 $v_t = \beta v_{t-1} + (1 - \beta) g_t^2$ ,  $\theta_{t+1, i} = \theta_{t, i} - \frac{\alpha}{\sqrt{v_{t, i} + \epsilon}} g_{t, i}$  ♦  $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$  ♦  
 $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ ,  $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$  ♦  
 $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$  ♦  
 $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$  ♦  
 $\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \lambda \theta_{t-1}$  ♦  
 $\theta_t = \theta_{t-1} - \alpha \text{sign}(\hat{m}_t)$  ♦  
 $\theta_t = \theta_{t-1} + \alpha \arctan(\hat{m}_t)$  ♦  
 $\theta_{\text{tgt}} \leftarrow m \theta_{\text{tgt}} + (1 - m) \theta_{\text{onl}}$  ♦

**Init.**  $\text{Var}(W) = \frac{1}{n_{in}}$  (or  $\frac{2}{n_{in} + n_{out}}$ ) ♦  
 $\text{Var}(W) = \frac{2}{n_{in}}$  ♦  
**Regularization.**  $\tilde{J} = J + \lambda \sum_i |w_i|$  ♦  
 $\tilde{J} = J + \frac{\lambda}{2} \sum_i w_i^2$  ♦  
 $\tilde{L} = L + \alpha_1 \sum_i |w_i| + \alpha_2 \sum_i w_i^2$  ♦  
 $w \leftarrow w - \eta (\nabla_w L_0 + \lambda w)$  ♦  $r_j \sim \text{Bernoulli}(p)$  ♦  
 $\bar{f} = \frac{1}{\sum_{i=1}^n \mathbb{1}(f(i) > 0)}$  ♦  $\bar{p} = \frac{1}{n} (r \odot h)$  ♦  
 $\mathbb{E}[y - \hat{f}]^2 = (\mathbb{E}[\hat{f}] - \bar{f})^2 + \text{Var}(\hat{f}) + \sigma^2_{\text{noise}}$  ♦  
 $\text{gap} = \text{Error}_{\text{val}} - \text{Error}_{\text{train}}$  ♦

**Normalization.**  $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$  ♦  
 $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$  ♦  $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$  ♦  
 $y_i = \gamma \hat{x}_i + \beta$  ♦  $\hat{\mu} \leftarrow (1 - \rho) \hat{\mu} + \rho \mu_B$  ♦

$z = \frac{1}{d} \sum_{i=1}^d x_i$ ,  $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$  ♦  
 $\text{LayerNorm}(x + \text{Sublayer}(x))$  ♦  
**CNN.**  $W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$  ♦  
 $H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - d(K-1) - 1}{S} \right\rfloor + 1$  ♦  
#params =  $(f_w \cdot f_h \cdot C_{\text{in}} + 1) C_{\text{out}}$  ♦  
 $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$  ♦  $s(i) = \sum_k f(i+k) g(k)$  ♦  
 $s(i) = \sum_k f(i+l) g(k)$  ♦  
 $s(i, j) = \sum_m \sum_n f(i+m, j+n) g(m, n)$  ♦  
 $s(i, j, d) = \sum_m \sum_n \sum_c f(i+m, j+n, d+c) g(m, n, c)$  ♦  
 $y_p = \sum_{c=1}^C w_c c, p$  ♦  $y = \mathcal{F}(x) + x$  ♦  
 $y = \mathcal{F}(x) + W \Delta x$  ♦  
 $C_{\text{filter}} = C_{\text{in}}$ ,  $C_{\text{out}} = N_{\text{filters}}$  ♦  
**RNN/LSTM/GRU.**  
 $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$  ♦  
 $o_t = \text{softmax}(W_{hy} h_t + b_y)$  ♦  $L = \sum_{t=1}^T L_t$  ♦  
 $\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$  ♦  
 $\frac{\partial h_t}{\partial W_h} = \frac{\partial f}{\partial W_h} + \frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_h}$  ♦  
 $\frac{\partial h_t}{\partial b_k} = \prod_{i=k+1}^t \frac{\partial h_{i-1}}{\partial b_{k-1}}$  ♦  $\text{out} = \sigma(g) \odot d$  ♦  
 $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$  ♦  
 $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$  ♦  
 $\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$  ♦  
 $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$  ♦  
 $o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$  ♦  $h_t = o_t \odot \tanh(c_t)$  ♦  
 $r_t = \sigma(W_r [h_{t-1}, x_t] + b_r)$  ♦  
 $z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$  ♦  
 $\tilde{h}_t = \tanh(W [r_t \odot h_{t-1}, x_t] + b)$  ♦  
 $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$  ♦  
 $c = h_t^{\text{enc}}$ ,  $h_0^{\text{dec}} = c$  ♦  $h_t = A_t h_{t-1} + B_t x_t$  ♦  
 $y_t = \tilde{C}_t h_t$  ♦

**Attention/Transformers.**  
 $Q = XW^Q$ ,  $K = XW^K$ ,  $V = XW^V$  ♦  
 $s_{ij} = q_i \cdot k_j$  ♦  $S = \frac{QK^T}{\sqrt{d_k}}$  ♦  
 $\alpha = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right)$ ,  $\sum_j \alpha_{ij} = 1$  ♦  
 $\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$  ♦  
 $o_i = \sum_j \alpha_{ij} v_j$  ♦  $\text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + M \right) V$  ♦  
 $M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$  ♦  
 $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$  ♦  
 $\text{head\_dim} = \frac{d_{\text{model}}}{h}$  ♦  
 $q_i = Q z_i$ ,  $k_i = K h_i$ ,  $v_i = V h_i$  ♦  
 $\text{scores} \in \mathbb{R}^n \times n$ ,  $O(n^2)$  ♦  $p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$  ♦

$P(w_{1:T}) = \prod_{t=1}^T P(w_t | w_{<t})$  ♦  
 $\mathcal{L} = -\sum_t \log P(w_t | w_{<t})$  ♦  $h = W_0 x + \frac{\alpha}{T} B A x$  ♦  
 $\Delta W = B A$ ,  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times k}$ ,  $r \ll d, k$  ♦  
 $M = u v^T \Rightarrow \text{rank}(M) = 1$  ♦  
 $\text{sim} = \hat{a} \cdot \hat{t}^T$ ,  $\hat{a} = x / \|x\|_2$  ♦  
 $\mathcal{L} = \|f(x_1) - f(x_2)\|^2$  ♦  $\mathcal{L} = \|x - \hat{x}\|^2$  ♦  
 $\mathcal{L} = \|g(x_{\text{ctx}}) - z_{\text{tgt}}\|^2$  ♦  
**PE/embed.**  
 $x_w \in \{0, 1\}^V$ ,  $(x_w)_i = 1[i = \text{idx}(w)]$  ♦  
 $e = E w$  ♦  $\hat{e} = \frac{1}{C} \sum_{c=1}^C e w_c$  ♦  
 $\hat{y} = \text{softmax}(W_{\text{out}} e)$  ♦  
 $e_{\text{king}} - e_{\text{man}} + e_{\text{woman}} \approx e_{\text{queen}}$  ♦  $z = e + p$  ♦

$PE(\text{pos}, 2i) = \sin \left( \frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right)$  ♦  
 $PE(\text{pos}, 2i+1) = \cos \left( \frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right)$  ♦  
 $\text{div\_term} = 10000^{-2i/d_{\text{model}}}$  ♦  $\text{PE} \in \mathbb{R}^{p \times d}$  ♦